Error-detecting and error-correcting codes

Section 5.4 of *Numbers, Groups and Codes*

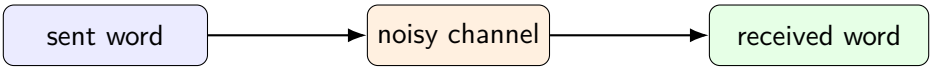
Applied Algebra
Instructor: Tuan Tran

Motivation and binary codes

Add redundancy so that errors become visible

The problem: messages can change in transit

Messages sent over a channel may be distorted.



digits may flip: $0 \leftrightarrow 1$

Examples: electrical noise, corrupted storage, and long-distance transmission.

Goal: detect errors, and when possible correct them without retransmission.

What these codes are not

There are two different coding goals.

Cryptographic codes

- keep information secret;
- decoding should be hard without a key;
- Chapter 1: public key codes.

Error-correcting codes

- keep information accurate;
- decoding should be systematic;
- Section 5.4: add check digits.

Here the issue is reliability, not secrecy.

Binary alphabet and binary words

From now on we work with binary information.

Let

$$\mathbb{B} = \mathbb{Z}_2 = \{0, 1\}$$

with addition and multiplication modulo 2.

A **word of length** n is a string of n binary digits.

Examples of words in $\mathbb{B}^4$:

$$0001, \qquad 1110, \qquad 0000.$$

We regard $\mathbb{B}^n$ as an Abelian group under componentwise addition.

Addition in $\mathbb{B}^n$

In $\mathbb{B}$, we have

$$1 + 1 = 0.$$

So every word is its own inverse:

$$w + w = 0.$$

Example:

$$010101 + 101000 = 111101.$$

A 1 in $v + w$ marks a position where v and w differ.

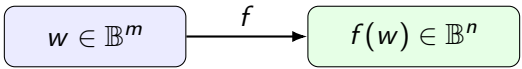
Coding functions

Suppose original messages are words of length m.

A **coding function** is an injective function

$$f : \mathbb{B}^m \longrightarrow \mathbb{B}^n, \qquad n > m.$$

Instead of sending w, we send $f(w)$.



The words in the image of f are called **codewords**.

Why f should be injective

If two different messages have the same codeword, they cannot be distinguished after transmission.

$$w_1 \neq w_2,$$
$$f(w_1) = f(w_2)$$

would mean the receiver cannot know whether w_1 or w_2 was sent.

So for a useful code:

$$f(w_1) = f(w_2) \quad \Rightarrow \quad w_1 = w_2.$$

Extra digits are allowed; loss of information is not.

Example 1: parity check code

Define

$$f : \mathbb{B}^m \longrightarrow \mathbb{B}^{m+1}, \qquad f(w) = wx,$$

where x is chosen so that $f(w)$ has an even number of 1s.

For $m = 3$:

w	000	001	010	011	100	101	110	111
$f(w)$	0000	0011	0101	0110	1001	1010	1100	1111

A single digit flip changes parity, so the received word is not a codeword.

What parity detects

The parity-check code detects:

$$1, 3, 5, \ldots \quad \text{errors.}$$

It does **not** detect:

$$2, 4, 6, \ldots \quad \text{errors.}$$

Example:

$$0000 \longrightarrow 1100$$

contains two flips, but both words have even parity.

Parity detects a problem but does not locate it.

Example 2: repeat three times

Define

$$f : \mathbb{B}^m \longrightarrow \mathbb{B}^{3m}, \qquad f(w) = www.$$

For example,

$$w = 101111$$

becomes

$$f(w) = 101111\,101111\,101111.$$

This code can detect one or two errors.

It can also correct one error by majority vote among the three blocks.

Majority vote correction

Suppose a word is sent as

$$www.$$

Break a received word into three blocks:

$$abc.$$

If at most one error has occurred, then two blocks agree with the original w.

Received blocks	a	b	c
No error	w	w	w
One error	w'	w	w
One error	w	w'	w
One error	w	w	w'

The repeated value is the most likely original word.

The trade-off

More check digits usually mean better reliability, but more transmission cost.

Code	Length sent	Detects	Corrects
Parity check	$m+1$	odd errors	none
Triple repetition	$3m$	1 or 2 errors	1 error

Later examples will do better than simple repetition.

Good codes spread codewords far apart while using few extra digits.

Weight

The **weight** of a binary word is the number of 1s in it:

$$\mathrm{wt}(w) = \text{number of 1s in } w.$$

Examples:

$$\mathrm{wt}(001101) = 3, \qquad \mathrm{wt}(000) = 0, \qquad \mathrm{wt}(111) = 3.$$

Equivalently, $\mathrm{wt}(w)$ counts the number of nonzero coordinates of w.

Distance between words

For $v, w \in \mathbb{B}^n$, define the distance between v and w by

$$d(v, w) = \mathrm{wt}(v - w).$$

Since in $\mathbb{B}^n$ we have $v - w = v + w$, this becomes

$$d(v, w) = \mathrm{wt}(v + w).$$

At each coordinate, $v_i + w_i = 1$ exactly when v_i and w_i differ.

Thus $d(v, w)$ is exactly the number of positions where v and w differ.

Examples:

$$d(010101, 101000) = \mathrm{wt}(111101) = 5,$$

$$d(1111, 0110) = \mathrm{wt}(1001) = 2.$$

Properties of distance

For $u, v, w \in \mathbb{B}^n$, the distance

$$d(v, w) = \mathrm{wt}(v + w)$$

satisfies:

$$d(v, w) \geq 0, \qquad d(v, w) = 0 \iff v = w,$$

$$d(v, w) = d(w, v),$$

and

$$d(u, w) \leq d(u, v) + d(v, w).$$

Thus d is a genuine distance function, or **metric**, on $\mathbb{B}^n$.

Also, the weight of a word is its distance from the zero word:

$$\mathrm{wt}(w) = d(w, \mathbf{0}).$$

Minimum distance of a code

Let $\mathcal{C}$ be the set of codewords.

The **minimum distance** is

$$d_{\min} = \min\{d(u, v) : u, v \in \mathcal{C},\ u \neq v\}.$$

Why this matters:

- one error moves a word distance 1;
- two errors move it distance at most 2;
- in general, k errors move it distance at most k.

Error control is geometry in the Hamming graph.

Theorem 5.4.1: Error detection

Suppose a codeword $c \in \mathcal{C}$ is sent, and the received word is r.

An error is **detected** if

$$r \notin \mathcal{C}.$$

An error is **not detected** if the received word is another codeword:

$$r \in \mathcal{C} \qquad \text{and} \qquad r \neq c.$$

Theorem. A code detects all errors of size at most k if and only if

$$d_{\min} \geq k + 1.$$

Why the theorem is true

Assume $c \in \mathcal{C}$ is sent and at most k errors occur.

Then the received word r satisfies

$$d(c, r) \leq k.$$

So detection fails exactly when r is another codeword:

$$r \in \mathcal{C}, \qquad r \neq c, \qquad d(c, r) \leq k.$$

Therefore, to detect all errors of size at most k, we need every two distinct codewords to be farther apart than k.

$$d(c_1, c_2) > k \qquad \text{for all distinct } c_1, c_2 \in \mathcal{C}.$$

Since distances are integers, this is the same as

$$d_{\min} \geq k + 1.$$

Theorem 5.4.2: Error correction

Suppose a codeword $c \in \mathcal{C}$ is sent, and the received word is r.

If at most k errors occur, then

$$d(c, r) \leq k.$$

To **correct** the error, the receiver must be able to determine which codeword was sent.

So correction is possible exactly when c is the **unique** codeword within distance k of r.

$$\{c' \in \mathcal{C} : d(c', r) \leq k\} = \{c\}.$$

In other words, no received word should be within distance k of two different codewords.

Theorem 5.4.2: Error correction

Theorem. A code corrects all errors of size at most k if and only if $d_{\min} \geq 2k + 1$.

Why?
Suppose a received word r is within distance k of two codewords c and c':

$$d(c, r) \leq k \qquad \text{and} \qquad d(c', r) \leq k.$$

Then by the triangle inequality,

$$d(c, c') \leq d(c, r) + d(r, c') \leq k + k = 2k.$$

So if

$$d_{\min} \geq 2k + 1,$$

this kind of ambiguity cannot happen.

Correction means there is a unique nearest codeword.

Detection and correction at a glance

If the minimum distance is $d_{\min}$, then the code detects up to

$$d_{\min} - 1$$

errors, and corrects up to

$$\left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$$

errors.

$d_{\min}$	detects up to	corrects up to
2	1	0
3	2	1
4	3	1
5	4	2

Quick checks:

1. Compute $d(01011, 11001)$.
2. A code has $d_{\min} = 4$. How many errors does it detect? How many does it correct?
3. Why can parity check detect a single error but not correct it?
4. In the triple repetition code, why can two errors lead to a wrong correction?

Linear codes

Use subgroup structure to simplify distance and decoding

Linear codes

Let $f : \mathbb{B}^m \to \mathbb{B}^n$ be a coding function.

The code is **linear** if its set of codewords

$$\mathcal{C} = \mathrm{im}(f) \subseteq \mathbb{B}^n$$

forms a subgroup of $\mathbb{B}^n$ under addition.

Since every word is its own inverse, it is enough to check:

$$u, v \in \mathcal{C} \quad \Rightarrow \quad u + v \in \mathcal{C}.$$

These are also called **group codes**.

Why linear codes are useful

In a general code, we may have to check all pairwise distances:

$$d(u, v) \quad (u \neq v).$$

In a linear code, the difference of two codewords is again a codeword:

$$u - v = u + v \in \mathcal{C}.$$

Therefore

$$d(u, v) = \mathrm{wt}(u + v) = \mathrm{wt}(u - v).$$

So distances between pairs reduce to weights of codewords.

Theorem 5.4.3: minimum distance by weight

Theorem. For a linear code, the minimum distance between distinct codewords is the minimum weight of a non-zero codeword:

$$d_{\min} = \min\{\mathrm{wt}(c) : c \in \mathcal{C},\ c \neq 0\}.$$

Proof idea.
a non-zero codeword c satisfies

$$d(c, 0) = \mathrm{wt}(c),$$

so $d_{\min} \leq \mathrm{wt}(c)$.
Also if $u \neq v$, then $u + v \neq 0$ is a codeword, so

$$d(u, v) = \mathrm{wt}(u + v) \geq \min \mathrm{wt}(c).$$

Generator matrices

Let $m < n$.

A **generator matrix** is an $m \times n$ matrix over $\mathbb{B}$ of the form

$$G = (I_m \; A),$$

where A has size $m \times (n - m)$.

It defines a coding function

$$f_G : \mathbb{B}^m \longrightarrow \mathbb{B}^n, \qquad f_G(w) = wG,$$

where w is treated as a row vector.

Because $G = (I_m \; A)$,

$$f_G(w) = (w \mid wA).$$

The first m digits recover the original word.

Example: computing a codeword

Let

$$G = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{pmatrix}.$$

Then $G = (I_2 \ A)$ and $f_G : \mathbb{B}^2 \to \mathbb{B}^5$.

For $w = 10$:

$$10G = 10110.$$

For $w = 01$:

$$01G = 01011.$$

The complete set of codewords is obtained by adding rows of G.

Example: a $\mathbb{B}^2 \to \mathbb{B}^5$ linear code

For

$$G = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{pmatrix},$$

we get:

w	$f_G(w)$
00	00000
01	01011
10	10110
11	11101

The non-zero weights are $3, 3, 4$.

Thus $d_{\text{min}} = 3$, so this code detects 2 errors and corrects 1 error.

Theorem 5.4.4: generator matrices give linear codes

Theorem. If G is a generator matrix, then f_G is a linear code.

Reason. For $v, w \in \mathbb{B}^m$,

$$f_G(v + w) = (v + w)G = vG + wG = f_G(v) + f_G(w).$$

Also

$$f_G(0) = 0.$$

Therefore the image of f_G is closed under addition and contains 0.

The codewords form a subgroup of $\mathbb{B}^n$.

Another generator matrix example

Let

$$G = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

Some codewords are:

w	000	001	010	011
$f_G(w)$	000000	001111	010101	011010
w	100	101	110	111
$f_G(w)$	100111	101000	110010	111101

The minimum non-zero weight is 2.

So this code detects 1 error but does not correct all single errors.

A better small example

The book gives a code $f : \mathbb{B}^3 \to \mathbb{B}^{10}$ with generator matrix

$$G = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}.$$

The minimum non-zero codeword weight is 5.

Therefore

$$d_{\min} = 5.$$

So it detects 4 errors and corrects 2 errors.

This uses 10 digits, whereas repeating a 3-bit word four times would use 12 digits and perform worse.

Group-theoretic view of errors

Let

$$\mathcal{C} \leq \mathbb{B}^n$$

be the subgroup of codewords.

Let e_i be the word with one 1 in position i and zeros elsewhere.

If $w \in \mathcal{C}$ is sent and an error occurs in position i, then the received word is

$$w + e_i.$$

The set of all words that could arise from a single error in position i is

$$e_i + \mathcal{C}.$$

Error patterns are coset representatives.

Two errors and general error patterns

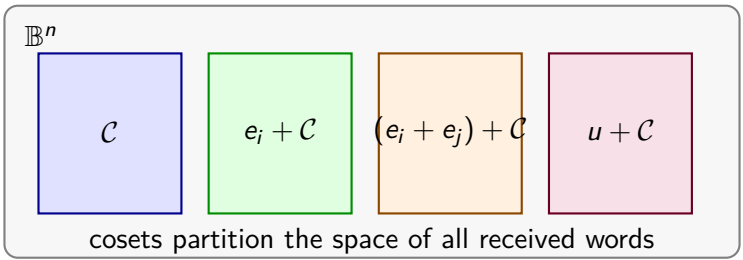
If errors occur in positions i and j, then the error pattern is

$$e_i + e_j.$$

The received word lies in the coset

$$(e_i + e_j) + \mathcal{C}.$$

More generally, every received word belongs to exactly one coset of $\mathcal{C}$ in $\mathbb{B}^n$.



Maximum likelihood decoding

When a word r is received, choose the nearest codeword.

Equivalently, in the coset containing r, choose an error pattern of least weight.

Definition. A **coset leader** is a chosen word of minimum weight in a coset.

If u is the coset leader for the coset containing r, then decode by

$$r \longmapsto r + u.$$

Because u represents the most likely error pattern.

Constructing a coset decoding table

Let $\mathcal{C}$ be the subgroup of codewords.

1. Put all codewords in the first row, with 0 first.
2. Choose a word of smallest weight not already in the table.
3. Use it as a coset leader and add it to every codeword.
4. Repeat until every word of $\mathbb{B}^n$ appears.

Each row is a coset of $\mathcal{C}$.

To decode a received word, find it in the table and replace it by the codeword at the top of its column.

Coset decoding table example

For the $\mathbb{B}^2 \to \mathbb{B}^5$ code with codewords

$$\mathcal{C} = \{00000, 01011, 10110, 11101\},$$

one possible standard array is:

00000	01011	10110	11101
00001	01010	10111	11100
00010	01001	10100	11111
00100	01111	10010	11001
01000	00011	11110	10101
10000	11011	00110	01101
10001	11010	00111	01100
00101	01110	10011	11000

Using the table

Suppose we receive

$$11100.$$

In the table, it lies below

$$11101.$$

So we decode

$$11100 \longmapsto 11101.$$

Since generator matrices are systematic, the original message is the first m digits:

$$11101 \longmapsto 11.$$

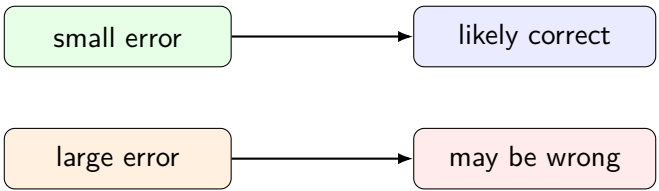
The table corrects by subtracting the row's coset leader.

What the table can and cannot promise

If $d_{\min} = 3$, all one-error received words are decoded correctly.

But words with two or more errors may be closer to a different codeword.

The last rows of a decoding table often correspond to larger error patterns.



Decoding is probabilistic in spirit: choose the most likely sent codeword.

Quick checks: Block 2

1. For the code with non-zero codeword weights $3, 3, 4$, find $d_{\min}$ and its error capability.
2. Explain why the image of a generator matrix is a subgroup of $\mathbb{B}^n$.
3. In a standard array, why do rows have equal size?
4. What does a coset leader represent physically?

Syndrome decoding

Store syndromes and coset leaders instead of the whole table

Why standard arrays can be too large

For a code $\mathbb{B}^m \to \mathbb{B}^n$:

- there are 2^m codewords;
- there are 2^n total received words;
- a full table has 2^{n-m} rows and 2^m columns.

Even moderate values of n become too large.

Instead of storing every word in every row, we store:

$$\text{syndrome} \quad \leftrightarrow \quad \text{coset leader}.$$

This gives the row information without the full row.

Parity-check matrix

Let

$$G = (I_m \; A)$$

be an $m \times n$ generator matrix.

The associated **parity-check matrix** is

$$H = \begin{pmatrix} A \\ I_{n-m} \end{pmatrix},$$

an $n \times (n - m)$ matrix.

For $w \in \mathbb{B}^n$, the **syndrome** of w is

$$wH \in \mathbb{B}^{n-m}.$$

Syndromes tell us which coset a received word belongs to.

Generator matrix versus parity-check matrix

The generator matrix describes the code directly:

$$\mathcal{C} = \{xG : x \in \mathbb{B}^m\}.$$

The parity-check matrix describes the same code by equations:

$$\mathcal{C} = \{w \in \mathbb{B}^n : wH = 0\}.$$

Thus G produces codewords, while H tests whether a word is a codeword.

G generates the code; H recognizes the code.

Theorem 5.4.5: syndrome test

Theorem. A word $w \in \mathbb{B}^n$ is a codeword if and only if

$$wH = 0.$$

Proof idea. A codeword has the form

$$w = (u \mid uA).$$

Then

$$wH = (u \mid uA) \begin{pmatrix} A \\ I_{n-m} \end{pmatrix} = uA + uA = 0$$

in $\mathbb{B}^{n-m}$.

The converse follows by reversing the equation.

Corollary 5.4.6: syndromes identify rows

Two words $u, v \in \mathbb{B}^n$ lie in the same row of the coset decoding table if and only if

$$uH = vH.$$

Reason:

$$u \text{ and } v \text{ same row} \iff u - v \in \mathcal{C}.$$

But

$$(u - v)H = uH - vH.$$

Since $x - y = x + y$ in $\mathbb{B}^{n-m}$, this is zero exactly when $uH = vH$.

One syndrome = one coset row.

Two-column decoding table

To decode with syndromes:

1. Compute the syndrome $s = wH$ of the received word w.
2. Find a minimum-weight word u satisfying

$$uH = s.$$

   This u is the coset leader for syndrome s.
3. Correct by adding the coset leader:

$$w \longmapsto w + u.$$

4. Read off the first m digits.

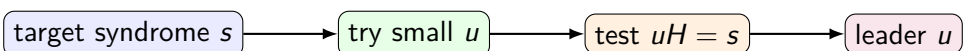
Coset leader = smallest error pattern with the given syndrome.

How to compute a coset leader

Given a syndrome s, search error patterns in order of increasing weight.

1. Try the zero error:
$$u = 00\cdots0.$$
2. Try all one-bit errors:
$$e_1, e_2, \ldots, e_n.$$
3. Try all two-bit errors:
$$e_i + e_j.$$
4. Continue until some u satisfies
$$uH = s.$$

The first such u is a coset leader.



Worked syndrome table: a $\mathbb{B}^3 \to \mathbb{B}^6$ code

Let

$$G = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}.$$

Then

$$H = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

The non-zero codewords have minimum weight 3, so the code detects 2 errors and corrects 1 error.

Syndromes and coset leaders

For the same $\mathbb{B}^3 \to \mathbb{B}^6$ code, one two-column table is:

Syndrome	Coset leader	Why?
000	000000	no error
001	000001	row 6 of H
010	000010	row 5 of H
100	000100	row 4 of H
110	001000	row 3 of H
011	010000	row 2 of H
101	100000	row 1 of H
111	100010	rows $1 + 5$

For example,

$$100010H = \text{row } 1 + \text{row } 5 = 101 + 010 = 111.$$